3. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_INPUT 100
#define MAX_HISTORY 20

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;
int history_position = 0;

/* Enable character-by-character input */
void enableRawMode(struct termios *oldSettings) {
    struct termios newSettings;

    tcgetattr(STDIN_FILENO, oldSettings);
    newSettings = *oldSettings;

    newSettings.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newSettings);
}

/* Restore normal terminal mode */
void disableRawMode(struct termios *oldSettings) {
    tcsetattr(STDIN_FILENO, TCSANOW, oldSettings);
}

/* Store command in history */
void addHistory(const char *command) {

    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    }
    else {
        for (int i = 1; i < MAX_HISTORY; i++) {
            strcpy(history[i - 1], history[i]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }

    history_position = history_count;
}

/* Clear current input line */
```

```
  GNU nano 8.7.1
/* Clear current input line */
void clearLine(int length) {

    for (int i = 0; i < length; i++) {
        printf("\b \b");
    }

    fflush(stdout);
}

/* Display recalled command */
void displayCommand(char *buffer, int *length) {

    clearLine(*length);

    printf("%s", buffer);
    fflush(stdout);

    *length = strlen(buffer);
}

/* Read command with arrow-key history */
int readCommand(char *buffer) {

    int position = 0;
    char ch;

    buffer[0] = '\0';

    while (1) {

        if (read(STDIN_FILENO, &ch, 1) <= 0)
            return 0;

        /* Enter key */
        if (ch == '\n' || ch == '\r') {

            buffer[position] = '\0';
            printf("\n");

            return 1;
        }

        /* Backspace */
        else if (ch == 127 || ch == 8) {

            if (position > 0) {
                position--;

                buffer[position] = '\0';

                printf("\b \b");
```

```
        /* Backspace */
        else if (ch == 127 || ch == 8) {

            if (position > 0) {
                position--;

                buffer[position] = '\0';

                printf("\b \b");
                fflush(stdout);
            }
        }

        /* Escape sequence */
        else if (ch == 27) {

            char seq1, seq2;

            if (read(STDIN_FILENO, &seq1, 1) <= 0)
                continue;

            if (read(STDIN_FILENO, &seq2, 1) <= 0)
                continue;

            /* Arrow keys */
            if (seq1 == '[') {

                /* UP ARROW */
                if (seq2 == 'A') {

                    if (history_count == 0)
                        continue;

                    if (history_position > 0)
                        history_position--;

                    strcpy(buffer, history[history_position]);

                    displayCommand(buffer, &position);
                }

                /* DOWN ARROW */
                else if (seq2 == 'B') {

                    if (history_count == 0)
                        continue;

                    if (history_position < history_count - 1) {

                        history_position++;
```

```
  GNU nano 8.7.1
                }

                /* DOWN ARROW */
                else if (seq2 == 'B') {

                    if (history_count == 0)
                        continue;

                    if (history_position < history_count - 1) {

                        history_position++;

                        strcpy(buffer, history[history_position]);

                        displayCommand(buffer, &position);
                    }
                    else {

                        history_position = history_count;

                        buffer[0] = '\0';

                        displayCommand(buffer, &position);
                    }
                }
            }
        }

        /* Normal character */
        else if (position < MAX_INPUT - 1) {

            buffer[position++] = ch;
            buffer[position] = '\0';

            printf("%c", ch);
            fflush(stdout);
        }
    }
}

int main() {

    char input[MAX_INPUT];
    struct termios oldSettings;

    printf("Dynamic Shell Input & Command History\n");
    printf("Use UP/DOWN arrows to recall commands.\n");
    printf("Type 'history' to view commands.\n");
    printf("Type 'exit' to quit.\n\n");

    enableRawMode(&oldSettings);
```

```
  GNU nano 8.7.1
int main() {

    char input[MAX_INPUT];
    struct termios oldSettings;

    printf("Dynamic Shell Input & Command History\n");
    printf("Use UP/DOWN arrows to recall commands.\n");
    printf("Type 'history' to view commands.\n");
    printf("Type 'exit' to quit.\n\n");

    enableRawMode(&oldSettings);

    while (1) {

        printf("shell> ");
        fflush(stdout);

        readCommand(input);

        /* Exit */
        if (strcmp(input, "exit") == 0) {
            break;
        }

        /* Display history */
        if (strcmp(input, "history") == 0) {

            printf("Command History:\n");

            for (int i = 0; i < history_count; i++) {
                printf("%d  %s\n", i + 1, history[i]);
            }

            continue;
        }

        /* Store command */
        if (strlen(input) > 0) {

            addHistory(input);

            printf("Command entered: %s\n", input);
        }
    }

    disableRawMode(&oldSettings);

    printf("Memory released successfully.\n");

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ nano Skill3a.c
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ gcc -Wall -Wextra Skill3a.c -o Skill3a
sanjana3027@DESKTOP-QJF5AGU:~/SKILL$ ./Skill3a
Dynamic Shell Input & Command History
Use UP/DOWN arrows to recall commands.
Type 'history' to view commands.
Type 'exit' to quit.
shell> shell> hello
Command entered: shell> hello
shell> Command entered: hello
Command entered: Command entered: hello
shell> mmand entered: lshell> ls
Command entered: shell> ls
shell> Cos
Command entered: Cos
shell> shell> pwd
Command entered: shell> pwd
shell> Command entered: pwd
Command entered: Command entered: pwd
shell> shell> history
Command entered: shell> history
shell> Command History:
Command entered: Command History:
shell> 1  hello
Command entered: 1  hello
shell> 2  ls
Command entered: 2  ls
shell> 3  pwd
Command entered: 3  pwd
shell>
```

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 2

/* Linked list node */
typedef struct Node {
    char *command;
    struct Node *next;
} Node;

/* Add a command to linked list */
void addNode(Node **head, const char *command) {

    Node *newNode = malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->command = malloc(strlen(command) + 1);

    if (newNode->command == NULL) {
        free(newNode);
        printf("Memory allocation failed.\n");
        return;
    }

    strcpy(newNode->command, command);

    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
    }
    else {
        Node *temp = *head;

        while (temp->next != NULL) {
            temp = temp->next;
        }

        temp->next = newNode;
    }
}

/* Display linked list */
void displayList(Node *head) {

    Node *temp = head;
```

```
/* Display linked list */
void displayList(Node *head) {

    Node *temp = head;

    printf("\nLinked List Commands:\n");

    while (temp != NULL) {
        printf("- %s\n", temp->command);
        temp = temp->next;
    }
}

/* Free linked list memory */
void freeList(Node *head) {

    Node *temp;

    while (head != NULL) {

        temp = head;
        head = head->next;

        free(temp->command);
        free(temp);
    }
}

int main() {

    int capacity = INITIAL_SIZE;
    int count = 0;

    /* Dynamic array allocation */
    char **commands = malloc(capacity * sizeof(char *));

    if (commands == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    Node *head = NULL;

    printf("Dynamic Memory Management Demo\n");
    printf("------------------------------\n");

    /* Commands to store */
    const char *input[] = {
        "ls",
        "pwd",
        "cd Documents",
```

```
 GNU nano 8.7.1

    /* Commands to store */
    const char *input[] = {
        "ls",
        "pwd",
        "cd Documents",
        "echo Hello",
        "gcc program.c"
    };

    int totalCommands = 5;

    for (int i = 0; i < totalCommands; i++) {

        /* Resize array when full */
        if (count == capacity) {

            capacity *= 2;

            char **temp = realloc(
                commands,
                capacity * sizeof(char *)
            );

            if (temp == NULL) {
                printf("Memory reallocation failed.\n");

                /* Free previously allocated memory */
                for (int j = 0; j < count; j++) {
                    free(commands[j]);
                }

                free(commands);
                freeList(head);

                return 1;
            }

            commands = temp;

            printf("\nBuffer resized to capacity: %d\n", capacity);
        }

        /* Allocate memory for command */
        commands[count] = malloc(strlen(input[i]) + 1);

        if (commands[count] == NULL) {
            printf("Memory allocation failed.\n");

            for (int j = 0; j < count; j++) {
                free(commands[j]);
            }
```

```
 GNU nano 8.7.1

        /* Allocate memory for command */
        commands[count] = malloc(strlen(input[i]) + 1);

        if (commands[count] == NULL) {
            printf("Memory allocation failed.\n");

            for (int j = 0; j < count; j++) {
                free(commands[j]);
            }

            free(commands);
            freeList(head);

            return 1;
        }

        strcpy(commands[count], input[i]);

        /* Add command to linked list */
        addNode(&head, input[i]);

        count++;

        printf("Stored command: %s\n", input[i]);
    }

    /* Display dynamic array */
    printf("\nDynamic Array Contents:\n");

    for (int i = 0; i < count; i++) {
        printf("%d. %s\n", i + 1, commands[i]);
    }

    /* Display linked list */
    displayList(head);

    /* Release dynamic array memory */
    for (int i = 0; i < count; i++) {
        free(commands[i]);
    }

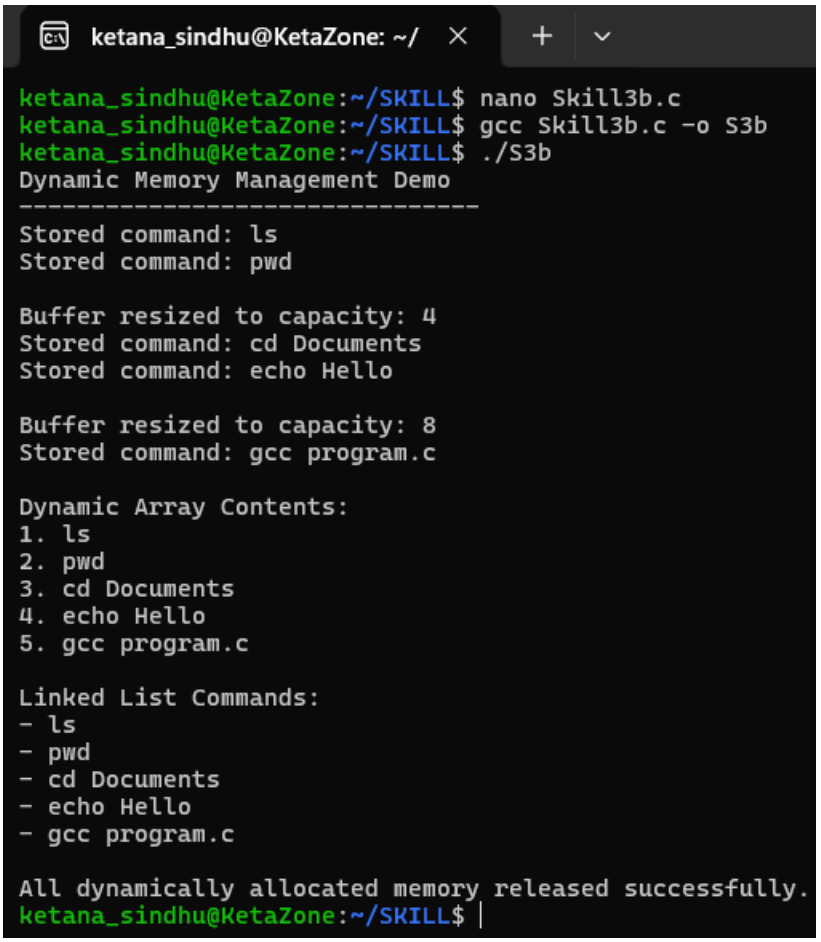
    free(commands);

    /* Release linked list memory */
    freeList(head);

    printf("\nAll dynamically allocated memory released successfully.\n");

    return 0;
}
```

Result:


ketana_sindhu@KetaZone: ~/
ketana_sindhu@KetaZone:~/SKILL$ nano Skill3b.c
ketana_sindhu@KetaZone:~/SKILL$ gcc Skill3b.c -o S3b
ketana_sindhu@KetaZone:~/SKILL$ ./S3b
Dynamic Memory Management Demo
Stored command: ls
Stored command: pwd
Buffer resized to capacity: 4
Stored command: cd Documents
Stored command: echo Hello
Buffer resized to capacity: 8
Stored command: gcc program.c
Dynamic Array Contents:
1. ls
2. pwd
3. cd Documents
4. echo Hello
5. gcc program.c
Linked List Commands:
- ls
- pwd
- cd Documents
- echo Hello
- gcc program.c
All dynamically allocated memory released successfully.
ketana_sindhu@KetaZone:~/SKILL$